University of Pisa

Parallel and Distributed Systems

Optimization of Partitioned Hash Join with Duplicates

Module 4 - Report

Gianluca Panzani

May 23, 2026

Contents

1 Implementation

This module extends the partitioned hash join with duplicates to a distributed-memory setting. The main executable is `hashjoin_mpi`; `hashjoin_hybrid` keeps the same MPI redistribution phase but uses OpenMP inside each rank for the local partition and join phases. The sequential and OpenMP implementations from the previous modules are kept as baselines, so every speedup is computed against the sequential execution with the same dataset type.

1.1 Executable Variants

The sequential executable `hashjoin_seq` is the correctness reference and the baseline for speedup. The OpenMP executable `hashjoin_omp` is the workload-balanced loop implementation from Module 3: in the selected records it uses 32 threads for the partition phase and 32 threads for the join phase, with guided scheduling.

The `hashjoin_mpi` executable follows a pure distributed-memory design. Each MPI rank generates a disjoint slice of the two input relations, redistributes its local records to the ranks that own the corresponding partitions, and then joins only the partitions assigned to it. The `hashjoin_hybrid` executable uses the same ownership and redistribution logic, but the local phases are parallelized with OpenMP. In the measured hybrid grid, the best records use 16 partition threads and 16 join threads per rank, guided scheduling, chunk size 4, and block size 32768.

1.2 Parameters

The main benchmark campaign uses $N_R = N_S = 50\,000\,000$, $P = 256$, `seed=13`, `max_key=1 000 000`, and three datasets: `uniform`, `skewed_90_5`, and `skewed_90_1`. The two skewed datasets concentrate 90% of the records in the keys mapped to a small subset of partitions, respectively about 5% and 1% of the total partition space.

The main comparison selects the best averaged record for each execution type and dataset, so some MPI records may use multiple ranks on the same node when this is faster. The dedicated scaling campaign is instead restricted to one active MPI rank per node: strong scaling keeps $N_R = N_S = 50\,000\,000$ fixed, while weak scaling increases the input size proportionally to the number of ranks.

2 Distributed Strategy

2.1 MPI Redistribution

The global algorithm remains the same as in the previous modules: records are assigned to partitions with the same mapping function, records belonging to the same partition are brought together, and each partition pair (R_p, S_p) is joined independently. The difference is that the owner of a partition is now an MPI rank.

Each rank first scans its locally generated records and computes the destination rank for each record. Three ownership policies are implemented: `block`, where contiguous ranges of partitions are assigned to ranks; `cyclic`, where partition p is assigned to rank $p \bmod R$; and `balanced`, where partition weights are estimated and assigned greedily to reduce rank imbalance. The selected best records in this campaign use `cyclic`, which is simple and effective for the tested skewed distributions because it spreads hot partition ids across ranks better than block ownership.

The redistribution itself is performed twice, once for R and once for S. First, `MPI_Alltoall` exchanges only the number of records that every rank will receive from every other rank. These counts are then used to allocate receive buffers and compute displacements. Then `MPI_Alltoallv` exchanges the actual records, allowing each sender-destination pair to use a different message size. This variable-size all-to-all exchange is necessary because the number of records mapped to each owner rank is not uniform, especially on skewed datasets.

2.2 Local Partition and Join

After redistribution, each rank owns complete global partitions. It locally partitions the received R and S records into the same $P = 256$ partition layout used by the sequential implementation. In the hybrid executable, this local partitioning uses the OpenMP blocked histogram/scatter implementation to reduce per-rank computation time.

The join phase is local after redistribution. Rank r scans only the partitions owned by r, builds a private flat count table for R_p, probes S_p, and accumulates a local `join_count`, `checksum1`, and `checksum2`. In the hybrid executable, large local partitions can be split into independent probe subranges using the workload-balanced OpenMP join decomposition from Module 3. Finally, `MPI_Reduce` sums the local outputs into the final global result on rank 0.

2.3 Measured Phase Times

The reported MPI and hybrid phase times are rank-level maxima, not rank-level averages. This is intentional: the wall-clock runtime is determined by the slowest rank. The measured local phases are `partition_time` and `join_time`. The redistribution/residual component includes the `MPI_Alltoall` and `MPI_Alltoallv` exchanges, reductions, barriers, and other synchronization overhead. Therefore, for MPI and hybrid runs, this component approximates the communication cost that must be paid before the local join can complete.

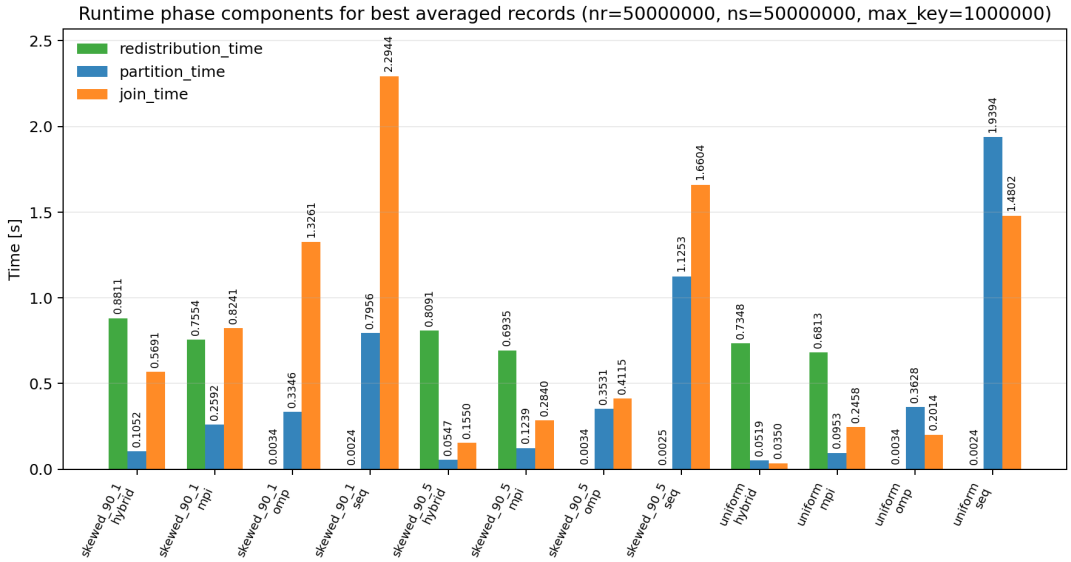


Figure 1: Runtime components for the selected records. The MPI and hybrid residual bars expose the cost of redistribution and synchronization, while the sequential and OpenMP bars are almost entirely local computation.

3 Averaged Results

Table 1 reports the best averaged total time per execution type and dataset. Each average is computed over five repetitions. The speedup is relative to the sequential baseline with the same dataset type. For sequential and OpenMP records, the MPI-related columns are only used to keep the table format uniform.

3.1 Uniform dataset

On the uniform dataset, OpenMP remains the fastest model. It reaches 0.568 s and 6.03× speedup because all communication is avoided and the partition/join work is still regular enough for a shared-memory implementation. Pure MPI and hybrid reduce the local computation substantially, but their communication and synchronization costs dominate the final runtime. For example, pure MPI spends

only 0.095 s in local partitioning and 0.246 s in local joining, but the redistribution/residual component is 0.681 s, which is already larger than the full OpenMP execution.

3.2 Skewed dataset

The hybrid version makes this trade-off even clearer. On uniform data it reduces local partitioning to 0.052 s and local joining to 0.035 s, but its total remains 0.822 s because the residual cost is 0.735 s. This means that, for this input size, adding OpenMP inside each MPI rank improves the local phases but cannot remove the all-to-all cost.

The skewed datasets change the balance. In `skewed_90_5`, OpenMP is still best: it takes 0.768 s, while MPI and hybrid take 1.101 s and 1.019 s. The local distributed join is faster, but not enough to amortize redistribution. In `skewed_90_1`, the join becomes much heavier: OpenMP spends 1.326 s in the join phase, while hybrid reduces the local join to 0.569 s and obtains the best total time among the parallel versions, 1.555 s. Therefore the hybrid approach becomes useful when the local join is expensive enough to compensate for part of the communication cost.

Dataset	Exec.	Nodes	MPI proc.	Ranks/node	Threads/rank	Strategy	Avg total [s]	Speedup
`uniform`	seq	1	1	1	1	block	3.422	1.00×
`uniform`	omp	1	1	1	32	block	0.568	6.03×
`uniform`	mpi	8	8	1	1	cyclic	1.022	3.35×
`uniform`	hybrid	8	8	1	16	cyclic	0.822	4.16×
`skewed_90_5`	seq	1	1	1	1	block	2.788	1.00×
`skewed_90_5`	omp	1	1	1	32	block	0.768	3.63×
`skewed_90_5`	mpi	1	8	8	1	cyclic	1.101	2.53×
`skewed_90_5`	hybrid	8	8	1	16	cyclic	1.019	2.74×
`skewed_90_1`	seq	1	1	1	1	block	3.092	1.00×
`skewed_90_1`	omp	1	1	1	32	block	1.664	1.86×
`skewed_90_1`	mpi	1	8	8	1	cyclic	1.839	1.68×
`skewed_90_1`	hybrid	8	8	1	16	cyclic	1.555	1.99×

Table 1: Best averaged records by execution type.

Dataset	Exec.	Total [s]	Local partition [s]	Local join [s]	Redistribution/residual [s]
`uniform`	seq	3.422	1.939	1.480	0.002
`uniform`	omp	0.568	0.363	0.201	0.003
`uniform`	mpi	1.022	0.095	0.246	0.681
`uniform`	hybrid	0.822	0.052	0.035	0.735
`skewed_90_5`	seq	2.788	1.125	1.660	0.003
`skewed_90_5`	omp	0.768	0.353	0.412	0.003
`skewed_90_5`	mpi	1.101	0.124	0.284	0.693
`skewed_90_5`	hybrid	1.019	0.055	0.155	0.809
`skewed_90_1`	seq	3.092	0.796	2.294	0.002
`skewed_90_1`	omp	1.664	0.335	1.326	0.003
`skewed_90_1`	mpi	1.839	0.259	0.824	0.755
`skewed_90_1`	hybrid	1.555	0.105	0.569	0.881

Table 2: Phase breakdown for the same selected records of Table 1.

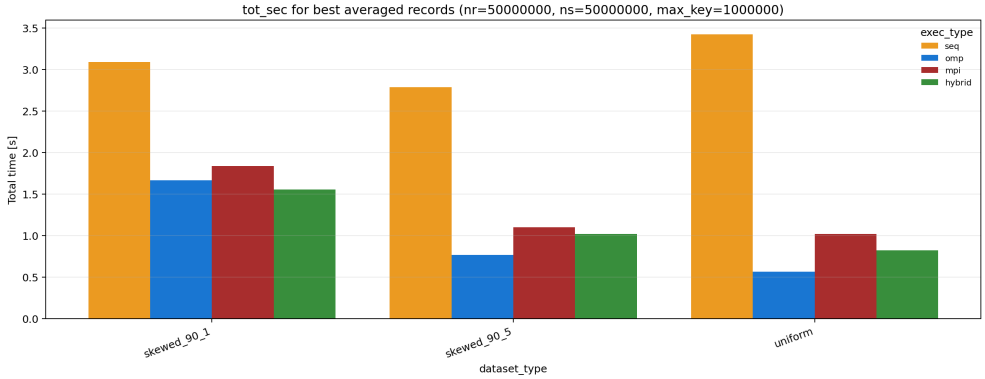


Figure 2: Best averaged total time for the main $50M + 50M$ benchmark.

4 Scalability Analysis

The scaling campaign uses the pure MPI implementation with one active rank per node. Weak scaling uses $10M + 10M$, $20M + 20M$, $40M + 40M$, and $80M + 80M$ records as the run grows from 1 to 8 ranks. Strong scaling keeps $N_R = N_S = 50\,000\,000$ fixed and varies the allocation from 1 to 8 nodes/ranks.

4.1 Weak Scalability

Weak scaling is best on the uniform dataset. The scaled speedup reaches about 8.1× at 8 nodes, because the input per rank remains almost constant while memory capacity and memory bandwidth grow with the number of nodes. This is the case where distributed memory is most effective: the all-to-all exchange is present, but it does not dominate enough to destroy scaling.

The skewed datasets scale much less. With `skewed_90_5`, the 8-node scaled speedup is about 2.2×; with `skewed_90_1`, it is about 1.9×. Even though the input size per rank is constant, increasing the global input also increases the absolute number of hot records and duplicate matches. This creates both more join work on hot partitions and a less balanced all-to-all exchange.

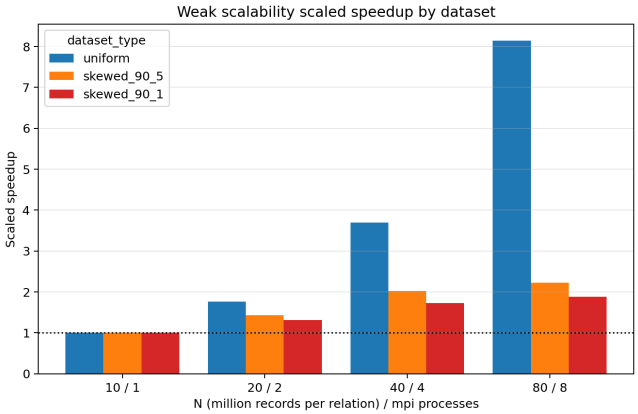
Compared with the Module 3 OpenMP weak-scaling campaign, MPI is clearly better on uniform data: the previous workload-balanced OpenMP curve stayed near 3× at large thread counts, while the distributed run reaches about 8.1× at 8 nodes. On skewed data the advantage is smaller. For `skewed_90_5`, the MPI result is close to the previous OpenMP peak; for `skewed_90_1`, MPI improves the largest configuration but still remains far from ideal because the hot partitions dominate the computation and communication pattern.

4.2 Strong Scalability

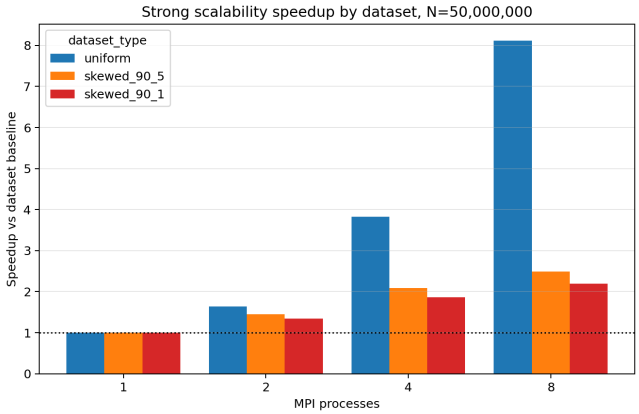
Strong scaling is also good on uniform data. The speedup reaches about 8.1× at 8 nodes, slightly above the ideal 8-rank line in the averaged data. This superlinear behavior is plausible for a memory-bound workload: when the input is split among more ranks, each rank handles smaller buffers and hash tables, which can improve cache behavior and reduce local memory pressure. The result should therefore be interpreted as a combination of parallelism and better locality, not as zero communication cost.

For skewed inputs, strong scaling is limited by hot partitions. `skewed_90_5` reaches about 2.5×, while `skewed_90_1` reaches about 2.2× at 8 ranks. Cyclic ownership helps by spreading hot partition ids across ranks, but the pure MPI version still assigns each complete partition to a single rank. If one or a few partitions contain most of the duplicate-heavy work, adding more ranks cannot split that work further.

The comparison with Module 3 explains the difference between shared-memory and distributed-memory scaling. On uniform data, MPI scales better because it distributes both memory footprint and bandwidth demand across nodes. On skewed data, the previous OpenMP workload-balanced join could split hot local work without paying an all-to-all redistribution cost, while MPI must first move records to partition owners. As a result, MPI is strongest on uniform data and less competitive on skewed inputs unless the local join is heavy enough for the hybrid version to compensate for part of the communication overhead.



Weak scaling scaled speedup.



Strong scaling speedup.

Figure 3: MPI scalability on 1, 2, 4, and 8 nodes/ranks. The left plot reports weak scaling with input size growing with the rank count, while the right plot reports strong scaling with fixed $50M + 50M$ input.

5 Correctness Validation

Correctness is checked at two levels. For small inputs, the implementations can compare their optimized output with a naive $O(N_R N_S)$ verifier. For benchmark-size inputs, where the naive verifier is too expensive, the checker compares the aggregated `join_count`, `checksum1`, and `checksum2` values across repeated runs and compatible implementations. The MPI and hybrid implementations use `MPI_Reduce` to aggregate exactly these three quantities, so the final output has the same form as the sequential baseline.

A benchmark record is accepted only if the three final values are consistent for the same dataset and configuration family. This keeps the validation step outside the measured region and avoids including checker overhead in the reported runtimes.

6 Conclusions

The MPI implementation preserves the original partitioned hash join structure while making the cost of distributed memory explicit. The main trade-off is clear: adding ranks reduces local partition and join work, but the all-to-all redistribution becomes a large fixed component of the runtime. On uniform data, OpenMP is still the fastest solution for the main $50M+50M$ benchmark because it avoids communication, while pure MPI is best in the dedicated scalability plots because it distributes memory traffic across nodes.

Skewness changes the conclusion. Moderate skew still favors OpenMP in the main averaged benchmark, because communication is not amortized. Strong skew increases the local join cost enough for the hybrid version to become the best parallel record among the compared implementations. The hybrid design is therefore the most promising distributed variant: MPI provides distributed memory capacity and partition ownership, while OpenMP reduces the cost of the local work assigned to each rank. The remaining limitation is that very hot partitions are still hard to balance perfectly, especially when redistribution and synchronization are on the critical path.